Grupo de Trabajo: Redes de Computadoras
Request for Comments: 9999 (ficticio)
Categoria: Experimental

J. Castillo Guzman
J. C. Gomez
ITCR

Noviembre 2026

FaroNet: Enlace por Luz (Faro) y Red Mesh Anonima
Version 1

Resumen

Este documento especifica FaroNet version 1: un protocolo de enlace que transmite tramas por luz infrarroja entre dos dispositivos llamados Faros (Capa 1), una red mesh que transporta mensajes sin revelar quien los origina (Capas 2 y 3), un formato de mensaje para chat anonimo (Capa 7) y la interfaz entre la mesh y una aplicacion publica desplegada en el edge. El diseno prioriza la minimizacion de metadatos: tamanos fijos, direcciones fisicas rotables, enrutamiento por capas de tres saltos y ausencia de registros.

Estado de este documento

Este documento no es un estandar de Internet. Es un trabajo academico del curso Redes de Computadoras de la Escuela de Ingenieria en Computacion del Instituto Tecnologico de Costa Rica. Se publica con fines de aprendizaje y su distribucion es libre.

El numero "RFC 9999" es ficticio y se usa solo para dar formato.

Aviso de derechos

Copyright (c) 2026 los autores. Se permite copiar y distribuir este documento sin modificaciones con fines academicos.

Tabla de contenidos

1. Introduccion
1.1. Terminologia
2. Capa 1: Protocolo del Faro
2.1. Medio fisico
2.2. Codificacion de linea
2.3. Formato de la trama
2.4. Campos de la trama
2.5. Verificacion de integridad
2.6. Tipos de trama
2.7. Direccion fisica
2.8. Acceso al medio y confiabilidad
2.9. Versionado
2.10. Tramas creadas fuera del Faro
2.11. Interfaz DispositivoLuzAdaptador
2.12. Medicion de desempeno
3. Capas 2 y 3: Red mesh FaroNet
3.1. Canales y transporte
3.2. HELLO: descubrimiento de vecinos
3.3. DIR: directorio de nodos
3.4. RUTA: enrutamiento por vector de distancias
3.5. Celdas de tamano fijo
3.6. Formato de la cebolla
3.7. Seleccion del circuito
3.8. Reenvio de celdas
3.9. Mensajes FaroNet sobre el enlace Faro
3.10. Gateways entre comunidades
4. Capa 7: Aplicacion anonima
4.1. Formato del mensaje
4.2. Mensajes privados
4.3. Temas privados
4.4. Mensajes publicos
5. Interfaz con el edge
5.1. Generalidades
5.2. Autenticacion del gateway
5.3. Salas publicas
5.4. Feed de temas
5.5. Buzon muerto
5.6. Directorio publico
5.7. Limite de peticiones
5.8. Politica de no registro
6. Boton de panico
7. Consideraciones de privacidad
7.1. Modelo de amenazas
7.2. Vigilancia y correlacion
7.3. Identificacion
7.4. Almacenamiento y retencion
7.5. Lo que ve el proveedor del edge
7.6. Divulgacion secundaria
8. Consideraciones de seguridad
8.1. Criptografia
8.2. Integridad del enlace
8.3. Suplantacion de vecinos
8.4. Denegacion de servicio
8.5. Ataques de Sybil
8.6. Repeticion
8.7. Robo del dispositivo

8.8. Seguridad del cableado
9. Consideraciones IANA
10. Referencias
10.1. Referencias normativas
10.2. Referencias informativas
Apendice A. Vectores de prueba
Apendice B. Mediciones de desempeno
Apendice C. Estado de implementacion y validacion
Agradecimientos
Direcciones de los autores

1. Introduccion

FaroNet es una red construida desde el cobre hacia arriba. Cada grupo suelda su propio dispositivo de transmision por luz (el Faro), monta sobre el una red mesh que oculta el origen de los mensajes y la conecta con Internet a traves de funciones serverless en el edge.

La amenaza principal no es solo que alguien lea el contenido de un mensaje, sino que registre quien habla con quien, cuando y con que frecuencia. Por eso cada decision de este documento se justifica contra el modelo de amenazas de la Seccion 7.

La arquitectura general es:

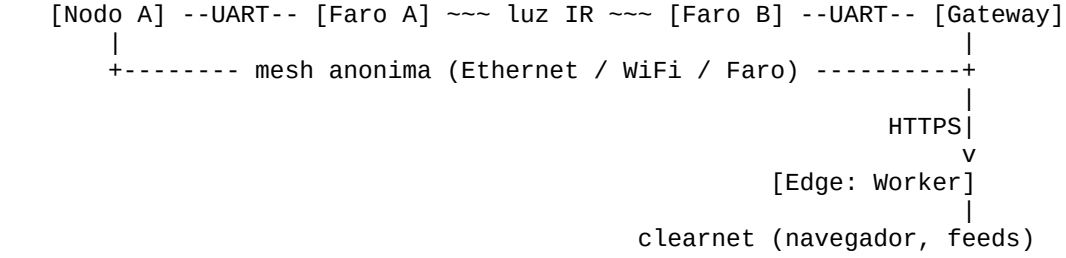


Figura 1: Arquitectura de FaroNet

En la implementacion de referencia cada Faro es una Raspberry Pi 2 con una placa perforada que contiene el emisor, el receptor, los LEDs indicadores y el boton de panico. La Raspberry corre el software del nodo, de modo que "nodo" y "Faro" comparten equipo.

1.1. Terminologia

Las palabras clave "DEBE", "NO DEBE", "REQUERIDO", "DEBERIA", "NO DEBERIA", "RECOMENDADO", "PUEDE" y "OPCIONAL" se interpretan como "MUST", "MUST NOT", "REQUIRED", "SHOULD", "SHOULD NOT", "RECOMMENDED", "MAY" y "OPTIONAL" de BCP 14 [RFC2119] [RFC8174] cuando aparecen en mayusculas.

Faro: dispositivo fisico que emite y recibe luz infrarroja y que se conecta al nodo por UART.

Trama: unidad de datos de la Capa 1, de 128 bytes como maximo.

Direccion fisica: identificador de 4 bytes del Faro, analogo a una direccion MAC, almacenado en memoria no volatil y rotable.

Nodo: programa que participa en la mesh. Se identifica con una llave publica Ed25519 [RFC8032] de 32 bytes.

Celda: unidad de 1024 bytes de tamano fijo que viaja por la mesh.

Cebolla: contenido cifrado por capas que va dentro de una celda. Cada relay quita una capa y solo conoce el siguiente salto.

Relay: nodo intermedio de una cebolla.

Gateway: nodo que conecta la mesh con el Worker en el edge, y que conecta dos comunidades a traves de un enlace Faro.

Worker: aplicacion publica desplegada en Cloudflare Workers.

Todos los enteros de varios bytes se transmiten en orden de red (big-endian). Los diagramas de formato usan la convencion de [RFC791]: cada fila representa 32 bits.

2. Capa 1: Protocolo del Faro

2.1. Medio fisico

El montaje de referencia usa Raspberry Pi 2 y electronica externa a 3.3 V. El LED IR D1 tiene R1 de 330 ohmios en serie y un driver NPN Q1 2N2222A. GPIO17 llega a la base por R2 de 2.2 kiloohmios; R3 de 10 kiloohmios une base y GND. El emisor de Q1 va a GND y su colector a la rama del LED. R1 PUEDE estar antes o despues de D1 si sigue en la misma rama serie sin derivaciones.

El receptor Q2 PT333 conecta emisor a GND y colector a RX. R4 de 10 kiloohmios une RX a 3.3 V. Sin luz RX esta alto y con luz suficiente baja. Se DEBERIAN obtener niveles cercanos a 3.3 V sin luz y menores que 0.5 V con luz; son objetivos de diseno, no mediciones ya realizadas. No hay comparador.

Los indicadores usan R5 y R6 de 1 kiloohmio. C1 de 100 nF une 3.3 V y GND como desacoplo. El conector es desmontable:

J1	Red	GPIO BCM	Pin fisico Raspberry
1	3V3	-	1
2	GND	-	6
3	TX	17	11
4	RX	27	13
5	LED TX	22	15
6	LED RX	23	16
7	PANICO	24	18

GPIO y receptor NO DEBEN alimentarse con 5 V. El esquema no incorpora la variante de emisor a 5 V. La Raspberry conserva su alimentacion normal por micro-USB; esto es independiente de los 3.3 V de la placa. Confirmar el patillaje del fabricante.

OOK enciende el LED con TX=1 y lo apaga con TX=0. El perfil --uart pigpio usa ondas de pigpiod en GPIO17 y un lector serie de GPIO27 con inversion, para recuperar UART 8N1. El reposo transmitido es 1 (LED encendido) y el receptor iluminado vale 0 fisicamente. El perfil conserva asi la polaridad UART logica.

Las dos puntas DEBEN usar los mismos baudios. Las herramientas conservan el valor CLI por defecto de 9600; este montaje se DEBERIA iniciar explicitamente con --baudios 300 y ajustar solo tras medir errores. Un loopback cercano no acredita otra placa ni otra distancia. La UART de hardware queda como transporte alternativo, no como el mapa de pines del circuito actual.

El LED propio y el receptor DEBEN aislarse opticamente. Para comunicar dos Faros cada LED mira al detector del otro Faro; solo en loopback local mira a su propio detector. Una entrada flotante o que no cambia con la luz NO DEBE tratarse como una recepcion funcional.

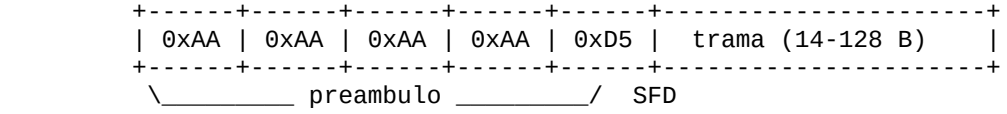
2.2. Codificacion de linea

Cada bit de la secuencia "preambulo + SFD + trama" se codifica con Manchester en dos medios bits:

```
bit 1  ->  medios bits 0, 1   (transicion de subida)
bit 0  ->  medios bits 1, 0   (transicion de bajada)
```

Los bits se recorren del mas significativo al menos significativo de cada byte.

El preambulo son 4 bytes 0xAA y el delimitador de inicio de trama (SFD) es 0xD5:



Los medios bits se empaquetan de 8 en 8 en bytes (primer medio bit en el bit mas significativo) y se envian por la UART. Una trama de 128 bytes produce (5 + 128) x 8 x 2 = 2128 medios bits, es decir 266 bytes de UART.

El receptor busca la secuencia de bits 0xAA 0xD5 despues de decodificar Manchester, probando los desfases 0 y 1 de medio bit. Un par de medios bits sin transicion (0,0 o 1,1) invalida la senal. Despues del SFD lee 12 bytes de cabecera, obtiene LEN del byte 11 y lee LEN + 2 bytes mas.

El receptor considera terminada una senal cuando la UART no entrega bytes durante un intervalo de silencio de al menos 20 tiempos de byte: unos 667 ms a 300 baudios y 21 ms a 9600. Las pruebas de rafagas DEBEN separar las tramas con una pausa mayor que ese silencio; la guia usa un segundo a 300 baudios.

SeriePigpio agrega un byte auxiliar 0x55 al final. Sus pares Manchester validos completan el transporte y no forman parte de LEN ni del CRC. Una trama maxima usa 267 bytes UART en este perfil (266 de senal mas uno auxiliar).

2.3. Formato de la trama

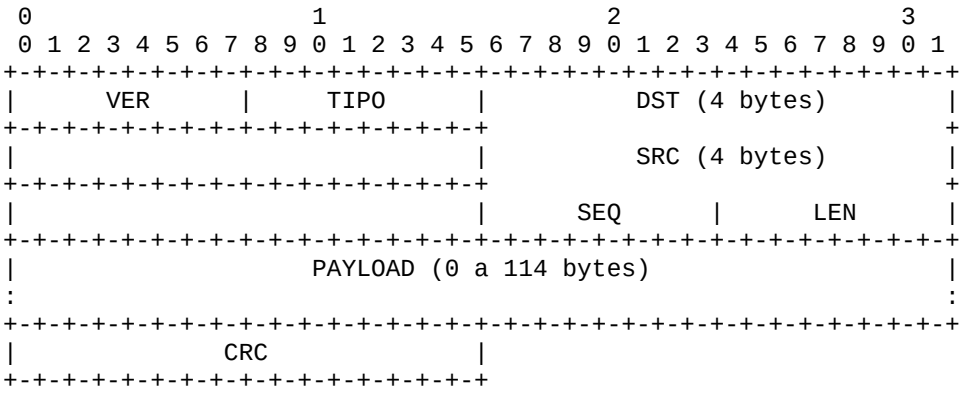


Figura 2: Trama del Faro v1

La cabecera mide 12 bytes, el CRC 2 bytes y el payload hasta 114 bytes, para un maximo de 128 bytes por trama.

2.4. Campos de la trama

VER (1 byte): version del protocolo de trama. Esta especificacion define la version 1.

TIPO (1 byte): tipo de trama (Seccion 2.6).

DST (4 bytes): direccion fisica del destino, o 0xFFFFFFFF para difusion.

SRC (4 bytes): direccion fisica del emisor.

SEQ (1 byte): numero de secuencia de 0 a 255, que vuelve a 0 despues de 255. Un ACK repite el SEQ de la trama que confirma.

LEN (1 byte): cantidad de bytes del PAYLOAD. DEBE ser 114 o menos. Es el unico campo de tamano dinamico y su largo se declara aqui.

PAYLOAD (LEN bytes): datos de la capa superior.

CRC (2 bytes): CRC-16 sobre todos los bytes anteriores, desde VER hasta el ultimo byte del PAYLOAD (Seccion 2.5).

Un receptor DEBE descartar en silencio una trama si:

- mide menos de 14 bytes o mas de 128 bytes;
- VER es 0 o mayor que la version mas alta que soporta;
- el largo total no es igual a 12 + LEN + 2;
- el CRC no coincide.

Un receptor DEBE descartar tramas cuyo DST no sea su direccion fisica ni la direccion de difusion.

2.5. Verificacion de integridad

El CRC es CRC-16/CCITT-FALSE: polinomio 0x1021, valor inicial 0xFFFF, sin reflexion de entrada ni de salida y sin XOR final. El valor de comprobacion para la cadena ASCII "123456789" es 0x29B1.

El CRC detecta errores accidentales del canal. No protege contra un atacante que modifique la trama, porque cualquiera puede recalcularlo; la integridad contra atacantes la dan las capas superiores (Secciones 3 y 8).

2.6. Tipos de trama

TIPO	Nombre	Uso
1	DATA	Lleva datos. Si DST no es difusion, el receptor DEBE responder con un ACK.
2	ACK	Confirma una trama DATA. LEN = 0. SEQ igual al de la trama confirmada.
3	BEACON	Anuncio periodico de presencia, normalmente a difusion. No se confirma.

Los valores 0 y 4 a 255 estan reservados. Un receptor DEBE ignorar tramas de tipo desconocido.

2.7. Direccion fisica

La direccion fisica mide 4 bytes y se genera con un generador de numeros aleatorios criptografico. Los valores 0x00000000 y 0xFFFFFFFF estan reservados y NO DEBEN asignarse a un Faro.

La direccion se almacena en la memoria no volatil del Faro (en la implementacion de referencia, el archivo ~/.faronet/direccion.bin en la tarjeta microSD, con permisos 0600).

Para evitar el rastreo de un dispositivo a lo largo del tiempo:

- Un Faro DEBE poder rotar su direccion bajo demanda.
- Un Faro DEBERIA rotar su direccion al menos cada 24 horas y cada vez que arranca.
- Al rotar, el Faro DEBE sobrescribir la direccion anterior y DEBERIA emitir un BEACON con la nueva direccion para que el vecino actualice su tabla.

Con 4 bytes aleatorios, la probabilidad de colision entre dos Faros

de un mismo enlace es despreciable (1 en 2^32). Un Faro DEBE ignorar las tramas cuyo SRC sea su propia direccion: en un enlace optico casi siempre son reflejos de su propia luz.

2.8. Acceso al medio y confiabilidad

El enlace es half-duplex: los dos Faros comparten el turno de transmision. El acceso al medio es CSMA con espera exponencial aleatoria:

1. Antes de transmitir, el emisor verifica si el medio esta ocupado. Se considera ocupado si la UART recibio al menos un byte en los ultimos 20 tiempos de byte.

2. Si esta ocupado, espera un tiempo aleatorio uniforme entre 0 y 50 x 2^n milisegundos, donde n es el numero de intento (n se limita a 6), y vuelve al paso 1.

3. Si esta libre, transmite la trama completa.

La entrega confiable usa parada y espera (stop-and-wait):

- Despues de enviar una DATA, el emisor espera un ACK con el mismo SEQ y con SRC igual al DST original durante 1 segundo.

- Si no llega, retransmite la misma trama con el mismo SEQ, hasta 3 veces. Si se agotan, informa el fallo a la capa superior.

- El receptor DEBE enviar un ACK por cada DATA dirigida a el, incluso si es una retransmision, y DEBE hacerlo apenas la recibe, aunque el mismo este esperando el ACK de una trama propia. DEBERIA descartar duplicados comparando (SRC, SEQ) con la ultima trama entregada de ese SRC.

Las tramas de difusion no se confirman.

2.9. Versionado

El campo VER permite introducir nuevas versiones. Un emisor DEBE usar la version mas alta que soporten ambas puntas; en ausencia de negociacion usa la version 1. Una version futura PUEDE anunciar las versiones soportadas en el payload del BEACON. Un receptor DEBE descartar tramas con VER mayor al que soporta y NO DEBE intentar interpretarlas.

2.10. Tramas creadas fuera del Faro

La biblioteca DispositivoLuzAdaptador DEBE aceptar una trama ya construida como secuencia de bytes (por ejemplo, armada con Scapy) y transmitirla sin modificarla, siempre que cumpla el formato de la Seccion 2.3. La biblioteca DEBE validar largo, version y CRC antes de transmitirla y DEBE rechazarla si no los cumple.

2.11. Interfaz DispositivoLuzAdaptador

La biblioteca DispositivoLuzAdaptador es la unica interfaz entre el nodo y el Faro. Ofrece:

- enviar(dst, payload, tipo): envia una trama sin esperar ACK y devuelve el SEQ usado.

- enviar_confiable(dst, payload): envia una DATA y espera su ACK segun la Seccion 2.8. Devuelve verdadero si fue confirmada.

- enviar_crudo(trama_bytes): transmite una trama externa segun la Seccion 2.10.

- recibir(timeout): devuelve la siguiente trama DATA o BEACON dirigida al nodo, o nada si vence el tiempo.

- cambiar_direccion(nueva): usa una direccion fisica nueva y la anuncia con un BEACON a difusion (Seccion 2.7).

La biblioteca usa un objeto Medio con tres operaciones: enviar medios bits, recibir medios bits y consultar si el medio esta ocupado. Existen dos medios: MedioUART (puerto pigpio o UART del sistema) y MedioSimUDP (simulacion UDP sin hardware). El montaje actual selecciona --uart pigpio. La biblioteca no es segura para varios hilos: el nodo
la usa desde un solo hilo dedicado al enlace.

Mientras transmite, el nodo DEBE encender el LED TX (GPIO22). Al recibir una trama con CRC valido, el nodo DEBE encender el LED RX (GPIO23) durante al menos 50 ms.

2.12. Medicion de desempeno

Para cada enlace se DEBEN medir la tasa de transferencia efectiva y la tasa de error de bit (BER) a al menos tres distancias distintas (por ejemplo 1 cm, 2 cm y 5 cm) bajo luz ambiental normal:

- Tasa efectiva = bytes de PAYLOAD entregados y confirmados / tiempo total, incluyendo preambulo, ACKs y retransmisiones.

- BER = bits erroneos / bits transmitidos. Se mide enviando tramas DATA a difusion con payload conocido (por ejemplo, bytes de un generador pseudoaleatorio con semilla fija) y comparando bit a bit lo recibido. Las tramas perdidas cuentan como todos sus bits erroneos.

En el perfil pigpio una trama maxima ocupa 267 bytes de UART. A 9600 baudios son unos 278 ms para 114 bytes de payload: un limite teorico cercano a 410 bytes/s sin ACK ni silencios. No sustituye una medida. La implementacion incluye la herramienta "python3 -m faro.medir" (modos enviar, recibir, tasa y eco) que calcula ambas medidas. Los resultados se reportan en el Apendice B.

3. Capas 2 y 3: Red mesh FaroNet

3.1. Canales y transporte

La mesh funciona sobre tres canales:

- Ethernet y WiFi 802.11: cada mensaje FaroNet viaja en un datagrama UDP. El puerto lo elige cada nodo y se publica en el directorio.

- Faro: cada mensaje FaroNet se fragmenta en tramas DATA (Seccion 3.9).

Los mensajes de control son objetos JSON [RFC8259] codificados en UTF-8. Todos los mensajes llevan:

t (texto): tipo de mensaje: "hello", "dir", "ruta" o "celda".

v (entero): version del formato del mensaje. Esta especificacion define la version 1. Un receptor DEBE tratar la ausencia de "v" como version 1 y DEBE descartar mensajes con "v" mayor al que soporta.

Un receptor DEBE descartar en silencio cualquier mensaje mal formado: JSON invalido, campos ausentes, tipos de dato incorrectos o valores fuera de rango. Un mensaje mal formado NO DEBE interrumpir el procesamiento de los mensajes siguientes.

3.2. HELLO: descubrimiento de vecinos

Cada nodo envia un HELLO a sus vecinos y semillas cada 10 segundos:

```
{"t":"hello","v":1,"id":"<64 hex>","canal":"udp",
 "dir":"192.168.1.20:6001","ts":1760000000}
```

id: llave publica Ed25519 del nodo en hexadecimal (64 caracteres).

canal: "udp" o "faro".

dir: direccion de contacto: "IP:PUERTO" para udp, o la direccion fisica en 8 caracteres hexadecimales para faro.

ts: hora de envio en segundos Unix.

Un nodo que recibe un HELLO registra al emisor como vecino directo y como ruta de costo 1. El campo "canal" DEBE coincidir con el canal por el que llego el HELLO; si no coincide, se descarta. Un vecino que no envia HELLO durante 30 segundos se olvida, junto con las rutas que pasaban por el.

Un nodo NO DEBE registrar como vecino directo a un nodo del que solo conoce la entrada de directorio: un vecino es solo quien envio un

HELLO directamente.

3.3. DIR: directorio de nodos

Cada nodo publica una entrada de directorio firmada. La entrada es un objeto JSON con estos campos:

Campo	Tipo	Significado
v	entero	Version del formato de entrada (1).
id	texto	Llave publica Ed25519, 64 hex.
cifrado	texto	Llave publica X25519, 64 hex.
canal	texto	"udp" o "faro".
direccion	texto	"IP:PUERTO" o direccion fisica (8 hex).
creado	entero	Hora de creacion, segundos Unix.
ttl	entero	Segundos de validez, de 1 a 86400.
rol	texto	OPCIONAL. "gateway" si el nodo es gateway.

La forma canonica de una entrada es su JSON con las llaves en orden alfabetico, sin espacios (separadores "," y ":") y codificado en UTF-8. La firma es Ed25519 [RFC8032] sobre la forma canonica, con la llave privada correspondiente al campo id, y se transmite en hexadecimal (128 caracteres).

El mensaje DIR transporta una o mas entradas:

```
{"t":"dir","v":1,"entradas":[
   {"entrada":{...},"firma":"<128 hex>"}, ...]}
```

Por compatibilidad, un receptor DEBE aceptar tambien la forma de una sola entrada {"t":"dir","entrada":{...},"firma":"..."}.

Un receptor DEBE ignorar los campos que no conoce; como la firma cubre la entrada completa, no pueden alterarse. Un receptor DEBE validar cada entrada y descartar la que:

- tenga campos ausentes o de tipo incorrecto, un id o cifrado que no sean 64 hex, o una direccion que no corresponda a su canal;
- tenga un "rol" distinto de "gateway";
- tenga una version mayor a la soportada;
- tenga una firma que no verifique contra su propio campo id;
- haya vencido (ahora >= creado + ttl);
- tenga ttl mayor a 86400;
- tenga creado mas de 300 segundos en el futuro.

Propagacion: cada nodo DEBE reenviar a sus vecinos, cada 10 segundos, todas las entradas validas y vigentes que conoce, no solo la propia. Por el enlace Faro, que es mucho mas lento, el directorio se envia solo cuando cambia o cada 60 segundos. Si un nodo conoce dos entradas con el mismo id, conserva la de mayor "creado". Las entradas vencidas se borran. Cada nodo vuelve a firmar su propia entrada cuando llega a la mitad de su ttl.

Como la firma acompana siempre a la entrada, un nodo intermedio no puede alterarla sin que se note. El directorio PUEDE replicarse en un servidor no confiable (Seccion 5.6).

3.4. RUTA: enrutamiento por vector de distancias

Cada nodo mantiene una tabla destino -> (siguiente salto, costo). Cada 10 segundos envia su tabla a sus vecinos:

```
{"t":"ruta","v":1,"id":"<64 hex>",
 "tabla":{"<id destino>":2,"<id destino>":3}}
```

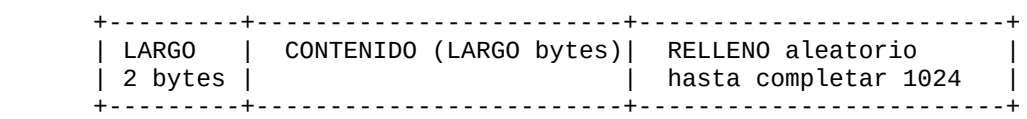
El costo es la cantidad de saltos. El valor 16 representa infinito (como en RIP [RFC2453]). Al recibir el anuncio de un vecino V con costo c hacia D, el nodo acepta la ruta "D por V con costo c + 1" si no tenia ruta a D, si la nueva es mas corta o si su ruta actual ya pasa por V. Las rutas que no se refrescan en 40 segundos se olvidan.

Un receptor DEBE descartar anuncios cuyo costo no sea un entero entre 0 y 16 y anuncios de nodos que no sean sus vecinos directos. La version 1 no implementa horizonte dividido; un enlace caido puede provocar conteo hasta infinito, limitado por el valor 16 y el vencimiento de 40 segundos.

La tabla de rutas se usa para reenviar celdas hacia el siguiente salto de la cebolla cuando este no es vecino directo (Seccion 3.8), en particular para cruzar de una comunidad a otra por el enlace Faro entre gateways.

3.5. Celdas de tamano fijo

Toda cebolla viaja dentro de una celda de exactamente 1024 bytes:

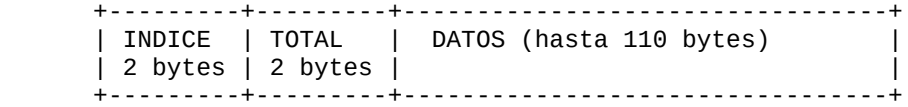


LARGO es la cantidad de bytes reales de CONTENIDO (como maximo 1022). El relleno DEBE generarse con un generador aleatorio criptografico para que no se distinga de datos cifrados.

Cada relay, despues de quitar su capa, DEBE volver a rellenar el

contenido interno a 1024 bytes. Asi, para un observador todas las celdas miden lo mismo en todos los saltos y no se puede deducir la posicion en el circuito ni el largo del mensaje.

Para el transporte, la celda se divide en fragmentos:



Cada fragmento mide como maximo 114 bytes, igual al payload maximo de una trama del Faro. Una celda produce 10 fragmentos. El receptor DEBE reensamblar solo cuando tiene todos los indices de 0 a TOTAL - 1, y DEBE descartar la celda si los fragmentos declaran totales distintos, si TOTAL es 0 o si algun INDICE es mayor o igual a TOTAL.

3.6. Formato de la cebolla

La cebolla usa sobres sellados (crypto_box_seal de libsodium) con las llaves X25519 publicadas en el directorio. Cada sobre agrega 48 bytes.

Capa del destino:

```
   sobre_destino = Sellar(llave_destino, 0x00 || MENSAJE)
```

Capa de cada relay, construida desde adentro hacia afuera:

```
   capa_i = Sellar(llave_relay_i, 0x01 || CABECERA || capa_interna)
```

La CABECERA mide 9 bytes:

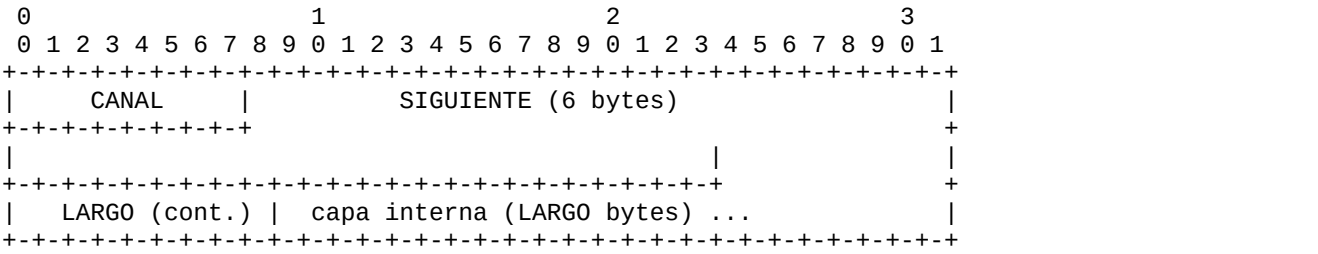


Figura 3: Cabecera de capa de relay

CANAL (1 byte): 0 = UDP, 1 = Faro.

SIGUIENTE (6 bytes): para UDP, 4 bytes de IPv4 y 2 bytes de puerto. Para Faro, los 4 bytes de la direccion fisica seguidos de 0x0000.

LARGO (2 bytes): largo de la capa interna.

El primer byte del texto descifrado identifica la capa: 0x00 para la capa del destino y 0x01 para una capa de relay. Los demas valores estan reservados.

Cada capa de relay agrega 1 + 9 + 48 = 58 bytes. Un mensaje de aplicacion de 256 bytes con tres relays produce una cebolla de 256 + 1 + 48 + 3 x 58 = 479 bytes, que cabe en una celda.

Un nodo que recibe una celda intenta abrirla como relay; si la capa es de destino, entrega el MENSAJE a la aplicacion. Si no puede abrirla, la descarta sin registrar nada.

3.7. Seleccion del circuito

Para enviar un mensaje, el origen elige tres relays distintos del directorio con un generador aleatorio criptografico. Ningun relay puede ser el propio origen ni el destino.

Un nodo DEBE usar al menos tres relays. Si el directorio tiene menos de cinco candidatos validos (sin contar origen y destino), el nodo DEBE rechazar el envio y avisar al usuario que el anonimato es insuficiente. Enviar sin relays solo PUEDE hacerse en pruebas, con una opcion explicita (en la implementacion de referencia, --permitir-directo), desactivada por defecto.

Con tres relays:

- el primer relay conoce al origen pero no al destino;
- el tercer relay conoce al destino pero no al origen;

- el relay del medio no conoce a ninguno de los dos.

Ningun nodo intermedio conoce a la vez el origen y el destino, salvo que los tres relays esten controlados por el mismo adversario.

3.8. Reenvio de celdas

Una celda viaja dentro de un mensaje CELDA:

```
{"t":"celda","v":1,"hacia":"<64 hex>","frags":["<base64>",...]}
```

hacia: id del nodo que debe abrir la siguiente capa de la cebolla.

frags: los fragmentos de la celda (Seccion 3.5) en base64.

Al recibir un CELDA:

- si "hacia" es su propio id, el nodo reensambla la celda y abre la cebolla (Seccion 3.6);
- si no, lo reenvia sin cambios: directo si "hacia" es vecino;

  si no, al siguiente salto de su tabla de rutas; y si tampoco hay ruta pero el directorio tiene una entrada UDP de "hacia", por UDP a esa direccion. Si nada de esto es posible, descarta el mensaje.

El relay que abre una capa obtiene CANAL y SIGUIENTE; busca en el directorio la entrada con esa direccion para obtener su id, que usa en "hacia". Si el siguiente salto es vecino directo, se le envia por su canal; si no, se reenvia por la tabla de rutas. Esto permite que un circuito cruce de una comunidad a otra por el enlace Faro: los nodos de reenvio solo ven "hacia", nunca el origen.

3.9. Mensajes FaroNet sobre el enlace Faro

Por el enlace Faro viajan los mismos mensajes JSON de las secciones 3.2 a 3.8. Como un mensaje puede medir mas que el payload de una trama, se fragmenta con el mismo formato de la Seccion 3.5 (INDICE, TOTAL, DATOS). Si el mensaje va a un vecino Faro conocido, cada fragmento viaja en una trama DATA confirmada (Seccion 2.8). Los HELLO, y los demas mensajes cuando no hay exactamente un vecino Faro, van a difusion sin confirmacion.

El emisor DEBE enviar todos los fragmentos de un mensaje, en orden, antes de empezar el siguiente. El receptor reinicia el reensamblado cuando recibe un fragmento con INDICE 0 y DEBE descartar un mensaje incompleto si pasan 10 segundos sin recibir el siguiente fragmento.

Los mensajes HELLO por el Faro usan canal "faro" y en "dir" la direccion fisica actual.

3.10. Gateways entre comunidades

Dos comunidades separadas se conectan con un par de gateways unidos exclusivamente por un enlace Faro:

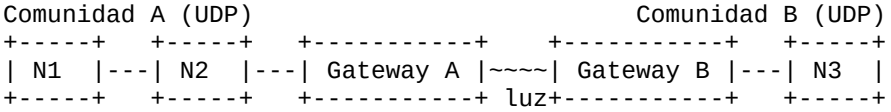


Figura 4: Dos comunidades unidas por luz

Cada gateway participa en las dos redes: anuncia por UDP las rutas que aprende por el Faro y viceversa. Asi las tablas de rutas de cada comunidad aprenden a llegar a los nodos de la otra pasando por su gateway.

4. Capa 7: Aplicacion anonima

4.1. Formato del mensaje

Los mensajes de la aplicacion miden exactamente 256 bytes:

VER	VIS	TEMA	HORA	LARGO	TEXTO	RELLENO
1 B	1 B	8 B	4 B	2 B	LARGO	aleatorio

VER (1 byte): version del formato (1).

VIS (1 byte): 0 = privado, 1 = publico.

TEMA (8 bytes): BLAKE2b de 8 bytes del nombre del tema en UTF-8. Para mensajes directos, BLAKE2b de 8 bytes de "dm:" seguido del id del emisor.

HORA (4 bytes): hora de creacion en segundos Unix.

LARGO (2 bytes): largo de TEXTO, hasta 240 bytes.

TEXTO: texto en UTF-8.

RELLENO: bytes aleatorios hasta completar 256 bytes.

El mensaje tiene un tiempo de vida de 3600 segundos. Un receptor DEBE descartar mensajes con HORA vencida o con HORA mas de 300 segundos en el futuro. Los mensajes recibidos se guardan solo en memoria y se borran al vencer.

4.2. Mensajes privados

Un mensaje privado es un mensaje con VIS = 0 enviado por una cebolla (Seccion 3.6) a la llave de cifrado del destinatario. Solo el destinatario puede leerlo (cifrado extremo a extremo).

El campo TEMA de un mensaje directo es una pista del emisor y NO esta autenticado en la version 1. Una aplicacion NO DEBE presentar esa pista como verificada.

4.3. Temas privados

Un tema privado tiene una llave simetrica de 32 bytes que conocen solo sus miembros. El mensaje de 256 bytes se cifra con crypto_secretbox (XSalsa20-Poly1305) y un nonce aleatorio de 24 bytes, y el resultado (296 bytes) se envia por cebolla a cada miembro.

La llave DEBE ser aleatoria y compartirse por un canal aparte, o derivarse de una frase secreta con Argon2id (crypto_pwhash) usando como sal los 16 primeros bytes de BLAKE2b("faronet-tema:" || nombre). La llave NO DEBE derivarse solo del nombre del tema, porque

cualquiera que adivine el nombre podria leer los mensajes. La implementacion de referencia usa Argon2id con los limites "interactive" de libsodium y exige frases de al menos 8 caracteres.

Los mensajes de temas privados nunca salen de FaroNet.

4.4. Mensajes publicos

Un mensaje publico tiene VIS = 1. Su TEXTO empieza con el nombre del tema:

```
+---------+---------------------+---------------------------+
| N (1 B) | NOMBRE (N bytes)    | texto del mensaje         |
+---------+---------------------+---------------------------+
```

N va de 1 a 32. NOMBRE solo PUEDE contener letras, numeros, "_" y "-" (la misma regla que aplica el Worker en sus URL). TEMA DEBE ser el BLAKE2b de 8 bytes de NOMBRE.

El origen envia el mensaje publico por cebolla a un nodo cuyo directorio indique "rol":"gateway". El gateway verifica que TEMA coincida con NOMBRE y lo publica en el Worker (Seccion 5.2). Si hay varios gateways, el origen elige uno al azar. El gateway no conoce al origen. Un gateway NO DEBE publicar mensajes con VIS = 0. Si el Worker no responde o responde 429 o 5xx, el gateway reintenta con espera creciente; si responde otro 4xx, descarta el mensaje.

5. Interfaz con el edge

5.1. Generalidades

El Worker se publica en Cloudflare Workers, plan gratuito, sobre HTTPS. Todas las respuestas de error son texto plano con el codigo HTTP correspondiente. Toda la configuracion vive en el archivo wrangler.jsonc del repositorio; los secretos se cargan con "wrangler secret put" y NO DEBEN subirse al repositorio.

Metodo	Ruta	Autentic.	Uso
POST	/api/relay	Firma GW	Publicar mensaje
POST	/api/directorio	Firma GW	Subir entradas
GET	/directorio	No	Leer directorio
GET	/sala/{tema}	No	Pagina de la sala
GET	/sala/{tema}/ws	No	WebSocket lectura
GET	/feed/{tema}.atom	No	Feed Atom
PUT	/buzon/{id}	No	Depositar blob
GET	/buzon/{id}	No	Retirar blob

5.2. Autenticacion del gateway

Las peticiones del gateway se firman con Ed25519. La llave publica del gateway se configura en el Worker como el secreto GATEWAY_PUB (64 hex). Cada peticion lleva tres encabezados:

```
X-FaroNet-Ts:     hora del gateway, segundos Unix
X-FaroNet-Nonce:  16 bytes aleatorios en hexadecimal
X-FaroNet-Firma:  firma Ed25519 en hexadecimal
```

La firma se calcula sobre la cadena:

```
"FARONET-GW-1" LF METODO LF RUTA LF TS LF NONCE LF
HEX(SHA256(CUERPO))
```

donde LF es el byte 0x0A y SHA256 es el resumen del cuerpo crudo.

El Worker DEBE rechazar con 401 la peticion si:

- falta algun encabezado o la firma no verifica;
- |ahora - TS| es mayor que 60 segundos;
- el NONCE ya se uso dentro de la ventana (el Worker guarda los nonces vistos durante 120 segundos y luego los borra).

Esto impide que un atacante repita una peticion capturada.

5.3. Salas publicas

Cada tema publico es una sala implementada con un Durable Object identificado por el nombre del tema. El cuerpo de POST /api/relay es:

```
{"v":1,"tema":"<nombre>","texto":"<texto>","hora":1760000000}
```

El Worker valida que "tema" tenga de 1 a 32 letras, numeros, "_" o "-", que "texto" no este vacio y mida como maximo 240 bytes, guarda el mensaje (Seccion 5.4) y lo entrega al Durable Object de la sala, que lo reenvia a todos los WebSocket conectados como {"id":N,"tema":"...","texto":"...","hora":N}. Un cuerpo que no sea JSON valido recibe 400.

GET /sala/{tema} devuelve una pagina HTML que se conecta por WebSocket a /sala/{tema}/ws y muestra los mensajes en tiempo real. Al conectarse, el WebSocket recibe primero {"historial":[...]} con los 20 mensajes vigentes mas recientes. La pagina se sirve con una politica de seguridad de contenido (CSP) restrictiva. Los WebSocket de navegadores son solo de lectura: el Durable Object DEBE ignorar los mensajes que envien. Solo el gateway publica.

5.4. Feed de temas

GET /feed/{tema}.atom devuelve un feed Atom [RFC4287] con las 20 entradas mas recientes del tema. El feed DEBE incluir los elementos id, title y updated, y cada entrada id, title, updated y content. Todo texto DEBE escaparse para XML (&, <, >, " y ').

Los mensajes publicos se guardan con un tiempo de vida de 7 dias y se borran al vencer (una tarea programada del Worker lo hace cada hora).

5.5. Buzon muerto

El buzon guarda blobs cifrados de tamano fijo, identificados por un identificador opaco que solo conocen las dos partes:

```
id = HEX(BLAKE2b-256(clave = secreto_compartido,
                     mensaje = "faronet-buzon-v1"))
```

- id DEBE medir exactamente 64 caracteres hexadecimales; si no, el Worker responde 400.
- El blob DEBE medir exactamente 1024 bytes. El cliente cifra con crypto_secretbox y rellena antes de enviarlo. El Worker responde 413 a cualquier otro tamano.
- El blob vive 24 horas.
- GET devuelve el blob y lo borra en la misma operacion. Una segunda lectura devuelve 404.

El Worker nunca recibe la clave ni el contenido en claro.

5.6. Directorio publico

El gateway sube cada 60 segundos las entradas validas del directorio con POST /api/directorio, con el cuerpo {"v":1,"entradas":[...]} en el formato de la Seccion 3.3. El Worker DEBE verificar cada firma (con la misma forma canonica) antes de guardarla y descartar las invalidas o vencidas; responde {"aceptadas":N,"rechazadas":M}. GET /directorio devuelve {"v":1,"entradas":[...]} con las entradas vigentes y sus firmas, para que cualquiera las verifique por su cuenta.

5.7. Limite de peticiones

El Worker DEBE limitar las peticiones a 60 por minuto por cliente en las rutas publicas y responder 429 al excederlo. El conteo se hace en memoria con el servicio de limite de peticiones de Cloudflare; la clave de conteo NO DEBE guardarse en almacenamiento persistente.

5.8. Politica de no registro

El Worker NO DEBE guardar ni escribir en bitacoras los encabezados CF-Connecting-IP, X-Forwarded-For, User-Agent ni ningun otro identificador del cliente. La observabilidad (logs) del Worker DEBE estar desactivada en wrangler.jsonc. Lo que el proveedor del edge sigue viendo se discute en la Seccion 7.

6. Boton de panico

Cada Faro tiene un boton fisico conectado a GPIO24 con resistencia de pull-up interna. Al presionarlo:

1. El nodo sobrescribe con ceros y borra los archivos de llaves y la direccion fisica del almacenamiento no volatil.

2. Borra de la memoria las llaves privadas, las llaves de temas, el directorio, la tabla de rutas y los mensajes recibidos.

3. Genera llaves nuevas (identidad Ed25519 y X25519) y una direccion fisica nueva, y la anuncia con un BEACON.

4. Hace parpadear los LEDs TX y RX juntos cinco veces a 5 Hz para indicar que el borrado termino.

El mismo borrado se puede pedir desde el chat con el comando /panico.

El nodo PUEDE volver a la red con una identidad nueva. Ningun dato anterior permite vincular la identidad nueva con la vieja.

7. Consideraciones de privacidad

Esta seccion sigue la estructura de [RFC6973].

7.1. Modelo de amenazas

Adversario	Que puede ver	Que no puede ver
Nodo malicioso de la mesh (relay)	Su vecino anterior y su siguiente salto; hora; que paso una celda de 1024 bytes.	Contenido. Origen y destino a la vez, salvo que controle los tres relays.
Observador del medio fisico (luz)	Que hay transmision, cuando y cuanto dura; direcciones fisicas de las tramas.	Contenido (va cifrado); identidad del nodo (la direccion rota).
Operador del gateway	Mensajes publicos que publica; volumen de trafico hacia el edge.	Origen de los mensajes publicos; mensajes privados.
Proveedor del edge	IP del gateway y de los navegadores; horas, tamanos; mensajes publicos; blobs cifrados; IP:puerto de los nodos UDP del directorio publico.	Contenido del buzon; origen de los mensajes dentro de FaroNet.

7.2. Vigilancia y correlacion

Las celdas de tamano fijo, el relleno aleatorio y los mensajes de 256 bytes impiden deducir el largo de un mensaje. Sin embargo, un adversario que observe a la vez la entrada y la salida de un circuito puede correlacionar por tiempo. La version 1 no agrega trafico de cobertura ni retrasos aleatorios; queda para versiones futuras.

7.3. Identificacion

Las direcciones fisicas rotan al menos cada 24 horas. Las identidades de la mesh son llaves que no contienen datos personales. Un nodo que conserva la misma llave por mucho tiempo puede ser rastreado dentro de la mesh por su id; un usuario que quiera evitarlo DEBERIA regenerar su identidad (boton de panico).

7.4. Almacenamiento y retencion

Ningun componente guarda bitacoras con direcciones IP, direcciones fisicas ni identificadores de usuario. Los nodos NO DEBEN imprimir ni registrar las direcciones de los saltos que reenvian. Los tiempos de vida son: mensajes de la app, 1 hora; entradas del directorio, hasta 24 horas; blobs del buzon, 24 horas; mensajes publicos en el edge, 7 dias; nonces, 120 segundos.

7.5. Lo que ve el proveedor del edge

Aunque el Worker no guarde encabezados, Cloudflare recibe cada conexion en su red antes de ejecutar el codigo. Por lo tanto ve la IP del gateway, la IP y el User-Agent de cada navegador que abre una sala o un feed, la hora y el tamano de cada peticion, y puede conservar esos datos segun sus propias politicas. El usuario DEBE asumir que el proveedor conoce la IP del gateway. El servicio de limite de peticiones usa la IP en memoria sin que el Worker la guarde.

7.6. Divulgacion secundaria

Los mensajes publicos son publicos por definicion: cualquiera puede leerlos, copiarlos y correlacionarlos. El usuario elige que es publico; los temas privados y los mensajes directos nunca salen de FaroNet.

8. Consideraciones de seguridad

8.1. Criptografia

FaroNet no implementa primitivas criptograficas propias. Usa libsodium (a traves de PyNaCl) en los nodos y WebCrypto en el Worker: Ed25519 para firmas, X25519 con XSalsa20-Poly1305 (sobres sellados) para la cebolla, XSalsa20-Poly1305 para temas y buzon, BLAKE2b para resumenes y Argon2id para derivar llaves de frases.

8.2. Integridad del enlace

El CRC del Faro solo detecta errores accidentales. Un atacante con un emisor IR puede inyectar tramas validas. Las capas superiores resisten esa inyeccion: las entradas de directorio van firmadas y las cebollas son autenticadas por los sobres; una celda alterada no se puede abrir y se descarta.

8.3. Suplantacion de vecinos

Los mensajes HELLO y RUTA no van firmados en la version 1. Un atacante en la red local puede anunciar rutas falsas o un vecino inexistente y atraer o descartar trafico (ataque de agujero negro). No puede leer ni alterar las celdas. Una version futura DEBERIA firmar HELLO y RUTA con la llave del nodo.

8.4. Denegacion de servicio

Un atacante puede saturar el medio fisico (luz) o la red UDP. Los nodos DEBEN descartar mensajes mal formados sin detenerse. El Worker limita las peticiones (Seccion 5.7) y el tamano de los blobs (Seccion 5.5).

8.5. Ataques de Sybil

Un adversario que publique muchas entradas en el directorio aumenta la probabilidad de controlar los tres relays de un circuito. La version 1 no lo previene. El minimo de cinco candidatos (Seccion 3.7) solo reduce el riesgo en redes pequenas.

8.6. Repeticion

Las peticiones al Worker tienen marca de tiempo y nonce (Seccion 5.2). Dentro de la mesh, un relay malicioso puede reenviar una celda dos veces; la aplicacion DEBERIA descartar mensajes repetidos. La implementacion de referencia guarda durante una hora el resumen BLAKE2b de cada mensaje recibido y descarta los que se repiten.

8.7. Robo del dispositivo

Si alguien roba un Faro encendido, puede leer las llaves de la microSD. El boton de panico solo protege si se presiona antes, y en una tarjeta microSD la sobrescritura no garantiza que el dato viejo desaparezca fisicamente. No hay cifrado de disco en la version 1.

8.8. Seguridad del cableado

Los pines GPIO de la Raspberry Pi toleran 3.3 V. El Faro DEBE alimentarse del pin de 3.3 V y NO DEBE conectarse a los pines de 5 V: con 5 V en el colector del fototransistor, RXD podria superar 3.3 V y danar el GPIO.

9. Consideraciones IANA

Este documento no requiere acciones de IANA.

10. Referencias

10.1. Referencias normativas

[RFC2119] Bradner, S., "Key words for use in RFCs to Indicate Requirement Levels", BCP 14, RFC 2119, March 1997.

[RFC8174] Leiba, B., "Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words", BCP 14, RFC 8174, May 2017.

[RFC8032] Josefsson, S. and I. Liusvaara, "Edwards-Curve Digital Signature Algorithm (EdDSA)", RFC 8032, January 2017.

[RFC8259] Bray, T., Ed., "The JavaScript Object Notation (JSON) Data Interchange Format", STD 90, RFC 8259, December 2017.

[RFC4287] Nottingham, M., Ed. and R. Sayre, Ed., "The Atom

Syndication Format", RFC 4287, December 2005.

10.2. Referencias informativas

[RFC791] Postel, J., "Internet Protocol", STD 5, RFC 791, September 1981.

[RFC2453] Malkin, G., "RIP Version 2", STD 56, RFC 2453, November 1998.

[RFC6973] Cooper, A., et al., "Privacy Considerations for Internet Protocols", RFC 6973, July 2013.

[RFC7322] Flanagan, H. and S. Ginoza, "RFC Style Guide", RFC 7322, September 2014.

[RFC9110] Fielding, R., Ed., et al., "HTTP Semantics", STD 97, RFC 9110, June 2022.

[TOR] Dingledine, R., Mathewson, N., and P. Syverson, "Tor: The Second-Generation Onion Router", USENIX Security Symposium, 2004.

[LIBSODIUM] "libsodium documentation", <https://doc.libsodium.org/>.

Apendice A. Vectores de prueba

Valores calculados con la implementacion de referencia.

CRC-16/CCITT-FALSE de "123456789":

0x29B1

Trama DATA, DST 0A0B0C0D, SRC 01020304, SEQ 7, payload "hola":

01 01 0A 0B 0C 0D 01 02 03 04 07 04 68 6F 6C 61 30 25

ACK correspondiente (DST 01020304, SRC 0A0B0C0D, SEQ 7):

01 02 01 02 03 04 0A 0B 0C 0D 07 00 79 A7

Primeros 32 medios bits de la senal de la trama DATA (preambulo 0xAA 0xAA en Manchester):

01100110 01100110 01100110 01100110

La trama DATA de 18 bytes produce (5 + 18) x 16 = 368 medios bits.

Una celda de 1024 bytes se divide en 10 fragmentos: 9 de 114 bytes y 1 de 38 bytes.

Un mensaje de 256 bytes en una cebolla de 3 relays mide 479 bytes.

TEMA del tema "futbol": 25 6F 65 61 90 71 DE B8.

Apendice B. Mediciones de desempeno

El enunciado exige BER y tasa efectiva a tres distancias bajo luz ambiental normal. No exige un metro salvo para el extra. La guia propone 1 cm, 2 cm y 5 cm, que DEBEN reemplazarse por las tres distancias reales usadas si se modifica el procedimiento.

Medidas fisicas al corte del 1 de noviembre de 2026: PENDIENTES. No se deduce BER cero ni tasa util del resultado de pytest.

Distancia	Sentido	BER	Tasa util (B/s)
1 cm	A -> B	PENDIENTE	PENDIENTE
1 cm	B -> A	PENDIENTE	PENDIENTE
2 cm	A -> B	PENDIENTE	PENDIENTE
2 cm	B -> A	PENDIENTE	PENDIENTE
5 cm	A -> B	PENDIENTE	PENDIENTE
5 cm	B -> A	PENDIENTE	PENDIENTE

Registrar fecha, luz, distancia, baudios, R4, cantidad de tramas, bits distintos, perdidas, tiempo y bytes confirmados. La BER reportada por medir incluye todos los bits de una trama perdida como erroneos: es una estimacion conservadora basada en tramas, no un instrumento que mida cada bit fisico UART.

A 300 baudios una trama maxima con el auxiliar tarda en teoria 267 * 10 / 300 = 8.9 s. Este calculo no es una medida de tasa. Fuente de resultados: docs/bitacora/pruebas.md.

Apendice C. Estado de implementacion y validacion

Se ejecutaron 140 pruebas Python y 23 pruebas locales del Worker; TypeScript se verifico sin errores. Son pruebas de software. El enlace fisico entre placas no quedo validado en el diagnostico del 1 de noviembre de 2026. Faltan voltajes y fotografias completas.

El grupo implemento el Worker pero no lo utilizo en la demostracion integrada. URL informada:

https://faronet-worker.faronet-jimmy-juan.workers.dev

El directorio publica direcciones de transporte para enrutar; esto expone metadatos aunque no se creen bitacoras historicas de usuarios. El borrado logico en microSD no garantiza eliminacion forense ni de todas las copias que puedan existir en memoria.

Agradecimientos

Al profesor Kevin Moraga por el enunciado del proyecto.

Direcciones de los autores

Jimmy Castillo Guzman
Instituto Tecnologico de Costa Rica
Centro Academico de Alajuela
Costa Rica

Juan Carlos Gomez
Instituto Tecnologico de Costa Rica
Costa Rica